PROGRAMMING 7311

POE Part 1

ABDULLAH ESSA
ST10442945

Table of Contents

Executive Summary/Introduction 3
Enterprise Framework Strategy (TOGAF) 4
Enterprise Application 6
Design Patterns 8
Selection and Justification 8
Observer Pattern 8
Chain of Responsibility Pattern 8
Decorator Pattern 8
Logic and Structure 9
UML Modelling 11
Scalability and Non-Functional Requirement Strategy 13
Availability Requirements 13
Interoperability Requirements 13
Scalability Strategy 14
Horizontal vs Vertical Scaling 14
Load Balancing 14
Caching 15
Conclusion 15
References 16

Executive Summary/Introduction

A global shipping coordinator known as “Techmove Logistics” has had significant issues caused by using outdated systems and legacy technology, such as data fragmentation, lost invoices, failure in contract compliance and workflow bottlenecks.

To solve this, a Global Logistics Management Service was proposed as an enterprise-grade solution to streamline the processing of service requests by allowing managers to request contracts and validates the requests, automatically convert international currencies, and create a centralised hub for contract information and status tracking. The system will use modern architecture to ensure high availability and scalability while being compatible across different devices and systems using Docker containerization.

This report is prepared for the Board of Directors of TechMove Logistics, who are tasked with guiding the company’s strategic technology investments. The audience is non-technical but highly invested in ensuring that new systems reduce operational risk, improve compliance, and support global expansion.

Enterprise Framework Strategy (TOGAF)

To align the Global Logistics Management System (GLMS) with TechMove's objectives, the project will adopt the TOGAF Architecture Development Method (ADM). TOGAF provides a structured, iterative process that integrates both business and technical requirements, ensuring scalability and stakeholder alignment. (Ardoq, 2024).

TOGAF was specifically chosen for this project as it balances stakeholder requirements with system design, ensures that every design decision ties back to TechMove's objectives and stakeholder needs and supports modular growth, so the GLMS can expand to handle more clients, contracts, and services without breaking.

This is in comparison to Zachman, which is primarily a classification schema that organizes architectural artifacts into perspectives, but it does not prescribe how to move from vision to implementation.

For this project, the first 4 phases (A-D) are the most critical:

A. Architecture Vision

The architecture of the system on a high level is defined, which should align with the goals of the organization and clearly explain the reasoning behind the architecture of the project (EA Learning, 2024). This phase also lists the stakeholders, securing their interest by giving more details as to how the project will work (Ardoq, 2024).

In this case, it will establish the scope and objectives of GLMS. This means a unified logistics platform that eliminates data fragmentation, ensures compliance, and supports global scalability.

B. Business Architecture

This is where the actual, detailed design of the business structure is developed, including its capabilities, structures and information flows (EA Learning, 2024). This means specific processes are defined, key assets are listed and a clear understanding of how the system will run is developed (Ardoq, 2024).

In this case, it will involve modelling the logistics processes that GLMS must support, including contract lifecycle management, service request validation, and compliance tracking to ensure that the system directly addresses TechMove's operational pain points and regulatory requirements.

C. Information Systems Architecture

Designing the Data Architecture (defining how data is collected, stored, integrated and used), and Applications Architecture (mapping business requirements into technical components) to ensure the system supports the

business architecture (EA Learning, 2024). Defining data models and laying out the planned application interactions is crucial in the step to ensure the end product is satisfactory (Ardoq, 2024).

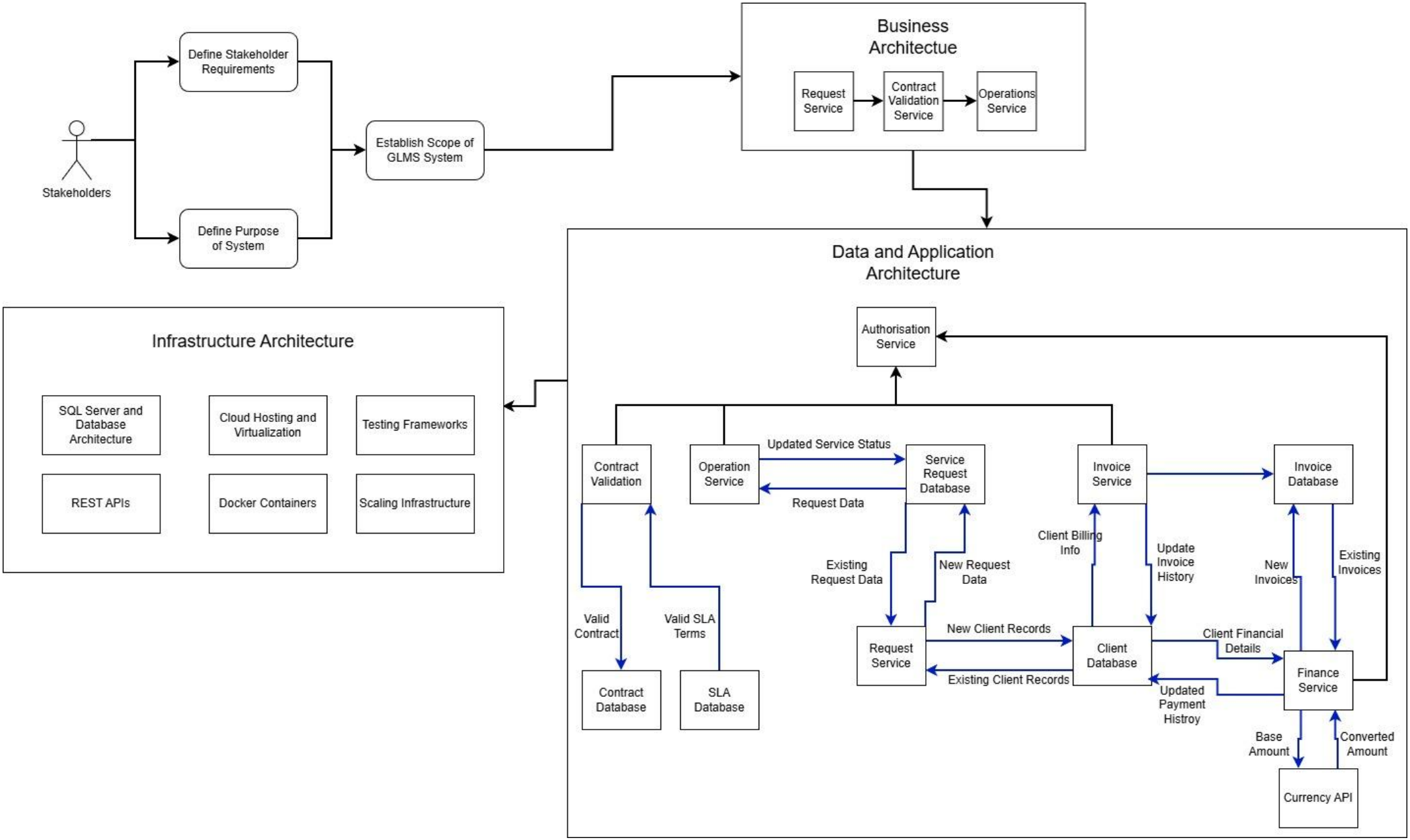
In this case, Contracts, clients, and service requests are modelled as core entities, while APIs enable real-time currency conversion and automated workflows. This phase ensures compatibility across devices and prepares the system for integration with external services, which is crucial when creating systems for existing businesses.

D. <u>Technology Architecture</u>

This is where the technical infrastructure (such as hardware, software, networking and other IT resources) for the system will be architected to ensure it supports the business and information architecture (EA Learning, 2024).

In this case, ASP.NET Core MVC will be used for the prototype, SQL Server for data and persistence, and containerized deployment via Docker for scalability, ensuring the system can handle large surges of users without losing performance.

Enterprise Application



The TOGAF diagram illustrates how the GLMS is structured across the four critical phases of the Architecture Development Method (ADM). Each block represents a layer of the enterprise architecture, showing how business requirements connect to technical design and infrastructure. Within the Data and Application Architecture block, the flow of data between services and databases is shown with blue arrows to show the direction and specific data of the transfer.

Design Patterns

A design pattern is a reusable solution to common software design problems to help with the creation of cleaner and maintainable systems (GeekForGeeks, 2026). The 3 design patterns that could be implemented in TechMove's system are the Observer, Chain of Responsibility and Decorator patterns.

Selection and Justification

Observer Pattern

A pattern which creates one-to-many relationships between subjects and observers in a subscription-based notification system. When a change occurs in the subject, all subscribers (observers) are notified and will be automatically updated (GeekForGeeks, 2026).

This pattern specifically helps with decoupling by separating a subject and its observers, where a subject does not know who or how many observers there are while allowing observers to be added or removed during runtime without affecting the subject (Dev Cookies, 2024). By applying the Observer pattern, the Notification Service can subscribe to relevant events and automatically push updates to the Logistics Manager without tightly coupling itself to the business logic of other services. This ensures real-time communication while maintaining modularity.

Chain of Responsibility Pattern

A behavioural pattern which allows a request to be passed along a chain of different handlers which either process the request or pass it onto the next handler (GeekForGeeks, 2026). This would work well for the validation of contracts in the system, as multiple validation checks can be chained together as handlers, where each handler performs a certain check on the contract before passing it to the next handler. This allows for a thorough validation process while keeping the system modular.

Decorator Pattern

A pattern which allows different behaviour to be added to objects dynamically, without modifying the original class or affecting other objects (GeekForGeeks, 2026). This would be useful in the validation of requests made by the logistics manager as different requests may need different validation layers, such as client details, SLA Terms or financial approval. Using this pattern, additional validation responsibilities can be added to a base request object depending on the situation while keeping modularity.

Together, these three patterns directly solve TechMove's decoupling and validation problems, ensuring the GLMS remains modular, extensible, and maintainable.

Logic and Structure

To ensure that the Global Logistics Management System (GLMS) meets TechMove's operational requirements, three design patterns have been selected: the Decorator Pattern, the Chain of Responsibility Pattern, and the Observer Pattern. Each pattern is implemented through a combination of interfaces, abstract classes, and concrete classes, with factories and dependency injection strategies ensuring flexibility at runtime. Together, these patterns establish a modular validation system and a decoupled notification system.

The Decorator Pattern is implemented within the request module to validate incoming requests, and so will be triggered each time a new request is made. An interface named IRequest defines the rules for all request objects, including a validate() method. A concrete class, BasicRequest, implements this interface and performs the mandatory checks required for every request. To extend validation dynamically, an abstract class AbstractRequestDecorator is introduced, which also implements IRequest and maintains a reference to another request object. Concrete decorators such as ClientCheckDecorator, DataFormatDecorator, and AuthorizationDecorator extend this abstract class, each adding specialized checks. A RequestValidationFactory determines which decorators should be applied to a given request, injecting them into the pipeline at runtime. The event trigger for this pattern is the submission of a new request, which instantiates a BasicRequest object and applies the relevant decorators. This design ensures that each request is validated against its own criteria, reducing errors and false flagging.

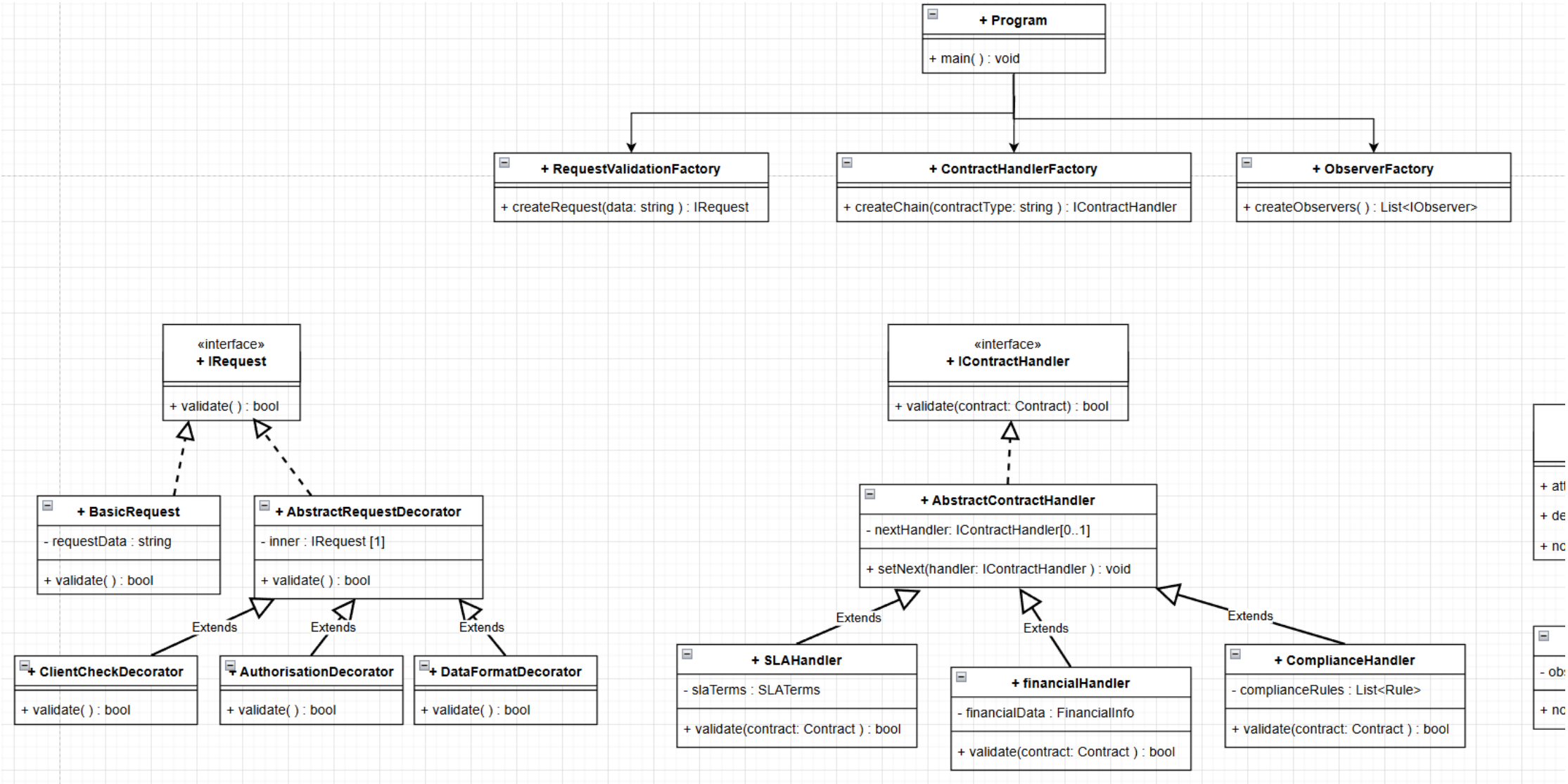
Once a request has been validated, the Chain of Responsibility Pattern is used to validate contracts. An interface named IContractHandler defines the validate(contract) method, while an abstract class AbstractContractHandler implements this interface and maintains a reference to the next handler in the chain. Concrete handler classes such as SLAHandler, FinancialHandler, and ComplianceHandler extend the abstract class, each responsible for a single validation step. The ContractHandlerFactory assembles the chain dynamically, injecting the appropriate sequence of handlers based on contract attributes. When a validated request triggers contract creation, the contract object is passed into the first handler. If the contract passes validation, it is forwarded to the next handler; if it fails, the chain terminates and an error is raised. This structure ensures modularity, as each handler is loosely coupled and only aware of its successor.

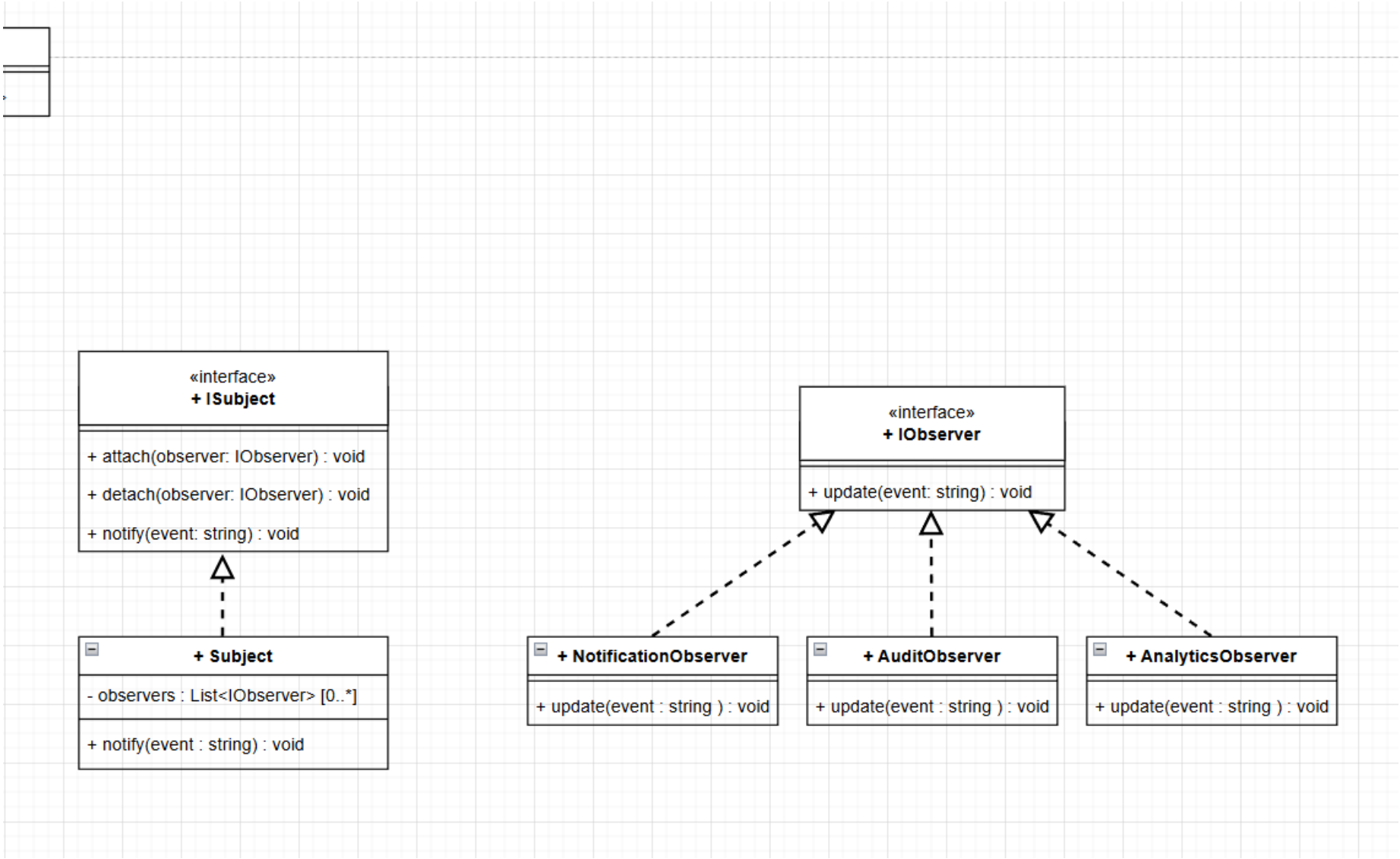
The Observer Pattern provides a decoupled mechanism for notifying stakeholders and subsystems of validation outcomes. An interface named ISubject defines methods for attaching, detaching, and notifying observers. A concrete Subject class implements this interface, maintaining a list of observers and triggering notifications when events occur. Observers are defined by the IObserver interface, which includes an update(event) method. Concrete observers such as NotificationObserver, AuditObserver, and AnalyticsObserver implement this interface, each handling

updates in a different way. An ObserverFactory is responsible for creating and injecting observers into the subject at runtime. Dependency injection ensures that the subject does not need to know the number or type of observers, maintaining loose coupling. The event trigger for this pattern is the outcome of request or contract validation, which prompts the subject to notify all registered observers simultaneously.

The Decorator Pattern ensures that requests are validated against both general and specific criteria. The Chain of Responsibility Pattern validates contracts through a sequential pipeline of independent checks. The Observer Pattern broadcasts the results of these validations to all interested parties without coupling them to the validation logic. By structuring the system around interfaces, abstract classes, concrete implementations, and factories, and by applying dependency injection at runtime, the GLMS achieves scalability, modularity, and alignment with TechMove's operational objectives.

UML Modelling




«interface»
+ ISubject
+ attach(observer: IObserver) : void
+ detach(observer: IObserver) : void
+ notify(event: string) : void
«interface»
+ IObserver
+ update(event: string) : void
+ Subject
- observers : List<IObserver> [0..*]
+ notify(event : string) : void
+ NotificationObserver
+ update(event : string) : void
+ AuditObserver
+ update(event : string) : void
+ AnalyticsObserver
+ update(event : string) : void

Scalability and Non-Functional Requirement Strategy

The Global Logistics Management System (GLMS) is designed to scale both functionally and operationally. Beyond the modularity provided by the Decorator, Chain of Responsibility, and Observer patterns, the system incorporates infrastructure strategies to handle surges in workload.

Availability Requirements

The GLMS must guarantee high availability to support global operations and contract surges. Availability requirements are defined as follows:

- **Uptime:** Minimum 99.9% uptime to ensure continuous logistics workflows.
- **Recovery Time:** System recovery must occur within as soon as possible in the case of a failure.
- **Recovery Point:** Constant recovery points should be created so that data loss does not impact data beyond a certain time limit (e.g a recovery point created every 5 minutes).
- **Redundancy:** All critical components (validation servers, contract handlers, notification services) must be deployed in clusters across multiple availability zones. This prevents single points of failure.
- **Failover:** Automatic failover must reroute traffic to healthy nodes if a server or data centre fails.

These requirements ensure uninterrupted operation even during hardware failures or traffic spikes

Interoperability Requirements

GLMS must integrate seamlessly with diverse systems and stakeholders across TechMove's global network. Interoperability requirements are defined as follows:

- **Common Interfaces:** All modules must expose APIs using REST to ensure a standard way of modules communicating with each other.
- **Data Format Compatibility:** Contracts, requests, and notifications must use standardized formats such as JSON and XML to ensure all systems can access and read the information.
- **Cross-Platform Support:** The system must run on both Windows and Linux servers, and be deployable in cloud environments (Azure, AWS) as well as on-premises.

- **Third-Party Integration:** GLMS must be able to operate with external services such as currency conversion APIs, payment gateways, regulatory compliance APIs, and analytics platforms.
- **Security Protocols:** All integrations must use TLS/SSL encryption and support the connection of additional security protocols for secure authentication and authorization.

These requirements ensure that GLMS can scale across regions, industries, and partner ecosystems without requiring custom rewrites.

Scalability Strategy

Horizontal vs Vertical Scaling

Vertical scaling is the expansion of resources of a single hardware or software component of the system (GeekForGeeks, 2026). This improves the performance of individual processes and makes it easier to manage but has physical and cost limitations and increases the possibility of downtime in the event of a failure as it is still a single server (GeekForGeeks, 2026).

Horizontal scaling focuses on the addition of more servers and nodes to increase the capacity and performance of a system (GeekForGeeks, 2026). This also increases the fault tolerance of the system as it prevents single points of failure, but increases complexity and adds latency due to communication between the nodes (GeekForGeeks, 2026).

The GLMS will focus more on horizontal scaling to increase the resilience of the system as a whole, but still make use of vertical scaling for database systems as each node must be able to handle large transactions of data at all times.

In order to make sure the system can implement horizontal scaling effectively, 2 main strategies will be used: Load balancing, and caching.

Load Balancing

Load balancing distributes network traffic across multiple servers in an efficient manner to avoid overloading any single nodes (IBM, 2026). There are 2 main techniques this system will make use of:

- **Round Robin -** Distributes the requests evenly across all servers.
- **Least Connections –** Routes new requests to the server with the fewer active connections.

During a surge of 50,000 contracts, load balancers distribute requests across multiple validation and contract-handling nodes, ensuring consistent performance.

Caching

Caching stores copies of data or files in a temporary storage location, allowing them to be accessed faster and reducing the load on the server (Fortinet, 2025).

This can be implemented in the system in multiple ways, such as:

- **Request Validation Cache:** Frequently used client identity and authorization checks cached.
- **Contract Metadata Cache:** SLA terms, compliance rules, and financial thresholds cached to accelerate handler checks.
- **Observer Notification Cache:** Frequently broadcast messages (e.g., “Contract Approved”) cached to avoid regenerating identical payloads for multiple observers.

This will be implemented by making use of a service called Redis as a dedicated caching layer. Redis (Remote Dictionary Server) allows data to be stored in the memory of the system, rather than on the storage drive of the user, allowing for faster, more reliable performance (IBM, 2026).

Conclusion

The scalability strategy for GLMS combines horizontal scaling, caching layers, load balancing, availability, and interoperability to deliver a resilient, high-performance system. In a case of a sudden influx of 50 000 new contracts, this system will accommodate every contract without sacrificing performance or reliability, ensuring no contracts are dropped or downtime is created.

References

Ardoq, 2024. *What Is TOGAF? Definition and Uses of This Enterprise Architecture Framework.* [Online]
Available at: https://www.ardoq.com/knowledge-hub/togaf
[Accessed 24 March 2026].

Dev Cookies, 2024. *Observer Design Pattern: A Complete Guide with Examples.* [Online]
Available at: https://devcookies.medium.com/observer-design-pattern-a-complete-guide-with-examples-ec40648749ff
[Accessed 26 March 2026].

EA Learning, 2024. *What is the TOGAF® Architecture Development Method?.* [Online]
Available at: https://www.ealearning.com/blog/what-is-the-togaf-architecture-development-method/
[Accessed 25 March 2026].

Fortinet, 2025. *What is Caching Data?.* [Online]
Available at: https://www.fortinet.com/resources/cyberglossary/what-is-caching
[Accessed 27 March 2026].

GeekForGeeks, 2026. *Chain of Responsibility Design Pattern.* [Online]
Available at: https://www.geeksforgeeks.org/system-design/chain-responsibility-design-pattern/
[Accessed 26 March 2026].

GeekForGeeks, 2026. *Decorator Design Pattern.* [Online]
Available at: https://www.geeksforgeeks.org/system-design/decorator-pattern/
[Accessed 26 March 2026].

GeekForGeeks, 2026. *Design Patterns Tutorial.* [Online]
Available at: https://www.geeksforgeeks.org/system-design/software-design-patterns/
[Accessed 26 March 2026].

GeekForGeeks, 2026. *Horizontal and Vertical Scaling | System Design.* [Online]
Available at: https://www.geeksforgeeks.org/system-design/system-design-horizontal-and-vertical-scaling/
[Accessed 27 March 2026].

GeekForGeeks, 2026. *Observer Design Pattern.* [Online]
Available at: https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/?utm_source=copilot.com
[Accessed 26 March 2026].

IBM, 2026. *What is load balancing?.* [Online]
Available at: https://www.ibm.com/think/topics/load-balancing
[Accessed 27 March 2026].

IBM, 2026. *What is Redis?.* [Online]
Available at: https://www.ibm.com/think/topics/redis
[Accessed 27 March 2026].